

Maven, teste de software e mock (a partir da pasta aula-07-mock)

A pasta

```
aula-07-mock/
├── pom.xml
└── src/
    ├── main/java/br/inatel/cdg/
    │   ├── BuscaInimigo.java      <- classe testada
    │   ├── Inimigo.java          <- POJO
    │   └── InimigoService.java    <- interface (a dependência)
    └── test/java/br/inatel/cdg/test/
        ├── InimigoConst.java      <- os JSONs hardcoded
        ├── MockInimigoService.java <- mock MANUAL
        ├── TesteBuscaInimigo.java  <- testes com o mock manual
        └── mockito/
            └── TesteBuscaInimigo.java <- os mesmos testes com Mockito
```

- Duas classes com o mesmo nome, em pacotes diferentes: é de propósito. A aula mostra o mesmo problema resolvido na mão e depois com framework
- src/main = código de produção, src/test = código que só existe para testar. Essa separação é do Maven, não do Java

Parte 1 — Maven

O problema que o Maven resolve

- Dependência: código externo que o meu sistema chama para funcionar (biblioteca, framework, pacote). Aqui são três: gson, junit, mockito
- Build: gerar a versão executável do sistema. Não é só compilar, é compilar + rodar teste + empacotar tudo (código, dependências, recursos) num arquivo só que roda em outra máquina
- Em Java o pacote é o .jar (Java ARchive). No Android é .apk
- Sem ferramenta de build eu teria que: baixar cada jar na mão, achar os jars que aquele jar precisa, colocar tudo no classpath, chamar javac com a lista inteira, chamar o runner de teste, e depois zipar. Toda vez. Em toda máquina
- Cada tecnologia tem a sua: Java -> Maven ou Gradle, JS -> npm/node, Android -> Gradle, Python -> pip com requirements.txt, C# -> MSBuild

- O requirements.txt do Python é o análogo mais direto do pom.xml: declaro o que preciso, a ferramenta resolve

Convenção sobre configuração

- Maven não pergunta onde está o código. Ele assume src/main/java, src/test/java, e joga a saída em target/
- Se eu seguir a convenção, o pom fica minúsculo. Se eu inventar minha própria estrutura, tenho que configurar tudo na mão
- É o princípio "convention over configuration". A decisão de engenharia por trás: padronizar reduz custo de manutenção e faz qualquer dev novo entender o projeto em 5 minutos

pom.xml

- Project Object Model. É o arquivo onde declaro tudo que quero que o Maven cuide
- Tem que estar na raiz do projeto e se chamar exatamente pom.xml

O pom da pasta:

```
<groupId>org.example</groupId>
<artifactId>aula_mock_b</artifactId>
<version>1.0-SNAPSHOT</version>
```

- groupId: identificação da empresa ou grupo de projetos, segue a convenção de nome de pacote Java (br.inatel.cdg, com.google.code.gson)
- artifactId: identificação do projeto
- version: versão do projeto
- Os três juntos são as coordenadas GAV. É o endereço único do artefato no repositório. Todo jar do mundo Java é encontrado por GAV
- SNAPSHOT -> versão em desenvolvimento, ainda muda. Sem SNAPSHOT -> versão liberada, imutável

```
<properties>
  <maven.compiler.source>16</maven.compiler.source>
  <maven.compiler.target>16</maven.compiler.target>
</properties>
```

- source: versão da linguagem que eu escrevo
- target: versão do bytecode gerado
- Serve para eu compilar com JDK novo mas gerar bytecode que roda em JVM antiga

Dependências e escopo

```
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.9.0</version>
</dependency>
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>4.4.0</version>
  <scope>test</scope>
</dependency>
```

- gson: converte String/instância Java em objeto JSON e vice-versa. É quem faz o parse do JSON dentro de Buscainimigo
- junit e mockito têm scope test. Isso quer dizer: só entram no classpath de compilação e execução dos testes, não vão para o jar final
- Faz sentido: o cliente não precisa do JUnit para jogar o jogo. Se eu esquecer o scope test, empacoto um framework de teste dentro do produto
- gson não tem scope, então usa o padrão compile: vale para tudo e vai para o jar. Correto, porque Buscainimigo (código de produção) importa gson
- Escopos: compile (padrão, vale em tudo), test (só teste), provided (compilo com ele mas quem fornece em produção é o servidor, ex.: API de servlet), runtime (não preciso para compilar, preciso para rodar, ex.: driver JDBC), system e import (raros)

Repositório central e repositório local

- As dependências ficam no Maven Central (mvnrepository.com é o site onde se procura o GAV)
- Na primeira build o Maven baixa e guarda em ~/.m2/repository. Da segunda vez em diante lê do disco
- Ou seja: rodar build offline funciona, desde que o jar já esteja no .m2
- Dependência transitiva: gson pode depender de outra coisa, e o Maven baixa a árvore inteira sozinho. Esse é o ganho maior sobre baixar jar na mão

- Conflito de versão: se duas dependências pedem versões diferentes da mesma lib, o Maven usa a mais próxima na árvore (nearest-wins). É origem clássica de bug de build que passa na minha máquina e quebra no servidor de CI

Ciclo de vida e fases

- O Maven não tem "comandos" soltos, ele tem ciclos de vida compostos de fases em ordem. Pedir uma fase executa todas as anteriores
- Ciclo default (o do build):

```
validate -> compile -> test -> package -> verify -> install -> deploy
```

- validate: valida se o projeto está correto e tem tudo que precisa
- compile: compila src/main/java em target/classes
- test: compila e roda os testes de src/test/java (quem roda é o plugin surefire)
- package: empacota em jar/war em target/
- verify: roda verificações de qualidade sobre o pacote
- install: copia o jar para o ~/.m2 local, para outros projetos meus usarem
- deploy: publica em repositório remoto compartilhado
- Ciclo clean: pre-clean -> clean -> post-clean. clean apaga target/
- Por isso o comando mais usado é mvn clean install ou mvn clean test: limpa e refaz do zero, sem restos de compilação antiga
- Consequência importante: mvn package roda os testes antes de empacotar. Teste quebrado -> build quebrada -> não gera jar. É assim que teste automatizado vira portão de qualidade

Comandos

mvn clean	apaga target/
mvn compile	compila só o main
mvn test	compila tudo e roda os testes
mvn package	gera o jar em target/
mvn install	instala o jar no repositório local (~/.m2)
mvn dependency:tree	mostra a árvore de dependências (bom para caçar conflito)

- O `:` significa plugin:goal. Fase é etapa do ciclo, goal é uma tarefa específica de um plugin. Fases são compostas de goals

Plugins

- Tudo que o Maven faz é plugin. compile é o maven-compiler-plugin, test é o maven-surefire-plugin, package é o maven-jar-plugin
- Quando quero um jar executável com as dependências dentro (fat jar), configuro o maven-assembly-plugin com jar-with-dependencies e digo qual é a mainClass
- Sem isso o jar gerado tem só as minhas classes e estoura NoClassDefFoundError procurando gson na máquina do cliente

Decisão: qual ferramenta de build

Critério	Ant	Maven	Gradle
Configuração	XML imperativo, eu escrevo cada passo	XML declarativo, por convenção	Groovy/Kotlin, script
Gerência de dependência	não tem nativa (precisa Ivy)	central, transitiva	central, transitiva
Flexibilidade	máxima	baixa, engessado	alta
Curva de aprendizado	média	baixa se seguir convenção	média
Build incremental	não	fraco	forte, cache

- Escolho Maven quando o projeto é Java padrão, a equipe é grande e eu quero previsibilidade
- Escolho Gradle quando preciso de build customizada, multi-módulo pesado ou Android
- Ant hoje só aparece em legado

Parte 2 — Teste de software

Para que serve teste

- Não é só para achar bug. É para garantir que a classe continua funcionando depois que o bug foi corrigido e depois de qualquer alteração futura (teste de regressão)
- Serve de documentação técnica: o teste registra qual comportamento é o esperado. Quando alguém quer saber o que o método faz, lê o teste
- Serve de rede de segurança para refatorar. Sem teste eu não mexo em código que funciona, com medo de quebrar
- Custo do defeito cresce muito conforme ele avança nas fases. Bug pego no teste de unidade custa pouco, bug pego em produção custa caro

Pirâmide de testes



sistema (poucos, lentos, caros, frágeis)

integração (médios)

unidade (muitos, rápidos, baratos)

- Base larga de teste de unidade, meio de integração, topo estreito de sistema
- Quanto mais alto, mais trabalhoso, mais lento e mais caro de manter
- Anti-padrão: pirâmide invertida (ou cone de sorvete). Acontece quando a equipe faz primeiro o que é mais fácil de imaginar, que é o teste manual pela interface. Resultado: suíte lenta, quebradiça e que não diz onde está o erro
- Raciocínio por trás: teste de unidade falha e aponta a linha. Teste de sistema falha e só diz "a tela deu erro"

Tipos de teste

- Unidade: testa uma unidade isolada. Em Java a convenção é que a unidade é uma classe (às vezes um método)
- Mock: teste de unidade de uma classe que tem dependência, substituindo a dependência por uma imitação. Continua sendo teste de unidade
- Integração: junta as peças de verdade, sem mock. Substitui o mock pelos objetos reais. É onde aparecem os problemas de contrato entre classes
- Sistema: testa o sistema inteiro, do ponto de vista do usuário, no ambiente montado. Selenium automatiza isso pelo navegador
- Aceitação: o cliente valida se o sistema faz o que ele pediu
- Regressão: rodar novamente a suíte para conferir que o que funcionava continua funcionando
- Funcional é o guarda-chuva: verifica se o sistema faz o que deveria fazer. Unidade, integração e sistema são todos funcionais
- Não-funcional testa o implícito: tempo de resposta, uso de CPU, uso de memória, carga, segurança, usabilidade
- Cai na prova a diferença: teste de mock NÃO é teste de integração. Aliás é o contrário, o mock existe justamente para não virar integração

Caixa preta e caixa branca

- Caixa preta: escrevo o teste olhando só a especificação, sem ver o código. Foco na entrada e na saída esperada. Técnicas: particionamento de equivalência, análise de valor limite
- Caixa branca (estrutural): escrevo olhando o código, tentando cobrir caminhos. Métricas de cobertura: comando, desvio, caminho

- Cobertura alta não significa teste bom. Dá para cobrir 100% das linhas sem uma única assertiva que preste

JUnit

- Framework de teste de unidade para Java. Entra como dependência com scope test
- Estrutura do método de teste e o porquê de cada parte:
 - public: para a dependência JUnit, que é código de fora, conseguir invocar o método por reflexão
 - void: o JUnit não espera resposta. Quem julga se passou é a assertiva, não o retorno. E isso reforça que um teste é independente do outro, ninguém devolve valor para ninguém
 - sem parâmetro: o JUnit não tem o que passar
- @Test é uma annotation, ou seja, um metadado. Não faz nada sozinha. Ela existe para outro software ler: o JUnit varre a classe procurando métodos marcados com @Test e chama cada um
- @Before roda antes de cada teste. É onde eu monto a fixture (o contexto do teste)
- Fixture: o cenário montado antes do teste rodar. Aqui é criar o mock e injetar em Buscalnimitigo
- O JUnit cria uma instância nova da classe de teste para cada @Test. Por isso um teste nunca enxerga o estado deixado pelo outro
- Outras anotações: @After (limpa depois de cada teste), @BeforeClass e @AfterClass (uma vez só, métodos static), @Ignore (pula), @Test(expected = Exception.class) (espera exceção)
- Assertivas: assertEquals, assertTrue, assertFalse, assertNull, assertNotNull, assertSame, assertEquals, fail
- assertEquals com double pede delta: assertEquals(200.0, valor, 0.1). Porque ponto flutuante tem erro de representação e comparar igualdade exata quebra

Padrão AAA

- Arrange, Act, Assert. Ou preparação, ação, verificação
- Nos testes da pasta: o @Before é o Arrange, a chamada de buscalnimitigo() é o Act, os assertEquals são o Assert
- Pode ter mais de uma assertiva por teste, quando fizerem sentido juntas. Aqui faz: eu quero verificar se a instância de Inimigo foi montada certa, então confiro nome, vida e arma. São três aspectos do mesmo comportamento
- O que não pode é o teste virar um script testando cinco comportamentos diferentes. Aí quando falha eu não sei o quê falhou

Nomenclatura que o surefire enxerga

- O maven-surefire-plugin, por padrão, só roda classes cujo nome bate com `Test.java`, `Test.java`, `Tests.java` ou `TestCase.java`
- `TesteBuscalnimigo` passa porque começa com "Test" (Test + eBuscalnimigo). Foi sorte do nome em português
- Se a classe se chamasse `BuscalnimigoTeste`, o `mvn test` rodaria e diria "no tests to run", sem erro nenhum. Pegadinha boa de prova

TDD

- Escrevo o teste primeiro, vejo ele falhar, escrevo o código até passar, refatoro. Red, green, refactor
- Duas motivações: garantir que o teste seja escrito (senão fica sempre para amanhã), e me colocar no papel de cliente do método antes de implementar, o que produz interface mais amigável
- Efeito colateral que interessa aqui: código nascido de TDD já nasce testável, ou seja, já nasce com dependência injetada

Parte 3 — Mock

O problema

- Muitas classes, para fazer o que fazem, precisam chamar outras classes
- Se eu incluir a dependência real no teste, eu saio do escopo do teste de unidade e viro teste de integração sem querer
- Pior quando a dependência é recurso externo: banco de dados, web service, servidor remoto. Aí o teste fica lento, precisa de ambiente configurado, depende da rede e falha por motivo que não tem nada a ver com a minha classe
- Durante teste de unidade é boa prática isolar a classe testada
- Objeto mock: instância simulada que imita o comportamento do objeto real de forma controlada
- Analogia da aula: teste de batida de carro com boneco. Além de não matar ninguém, o boneco é instrumentado, então eu consigo medir
- Quem cria o mock é quem escreve o teste, ou seja, o próprio dev da solução

Injeção de dependência

- Antes de mockar eu preciso que a classe aceite receber a dependência de fora. Esse é o pré-requisito

- Código escrito sem pensar em teste normalmente instancia a dependência dentro do método:

```
public Inimigo buscaInimigo(int id){  
    InimigoService service = new InimigoServiceReal(); // problema  
    String json = service.busca(id);  
    ...  
}
```

- Essa variável é local, não tem acesso externo, e a classe fica altamente acoplada. Não tem como o teste trocar esse objeto por um falso
- Três formas de injetar:
 - Construtor: passo a dependência ao instanciar a classe. É a forma usada na pasta
 - Setter: um método set que troca a dependência depois de instanciada
 - Parâmetro do método: passo a dependência na própria chamada
- Construtor é o preferido quando a dependência é obrigatória, porque garante que o objeto nunca existe num estado inválido. Setter serve quando a dependência é opcional ou pode mudar em tempo de execução
- Ponto importante da aula: parece trabalho a mais só para testar, mas não é. Ao injetar a dependência eu desacoplei a classe, ela passou a depender da interface e não da implementação. O código ficou melhor por causa do teste. Isso é engenharia de software, não burocracia de teste
- Formalmente isso é o princípio da inversão de dependência (o D do SOLID): módulo de alto nível não depende de módulo de baixo nível, os dois dependem de abstração
- E por isso InimigoService é uma interface. Se fosse classe concreta eu não teria onde encaixar o falso

O código da pasta, peça por peça

InimigoService — a dependência, declarada como contrato:

```
public interface InimigoService {  
    public String busca(int id);  
    public boolean inimigoExistente(int id);  
}
```

- Na vida real quem implementa isso vai numa API remota e devolve o JSON em String
- No teste quem implementa é o mock. A classe testada não sabe a diferença. É como um disfarce Inimigo — POJO, só estado:

```
public class Inimigo {
    private String nome;
    private double qtdVida;
    private String arma;
    // construtor + getters + setters
}
```

- POJO com só getter e setter não precisa de teste de unidade próprio. Não tem lógica para dar errado
- Ele acaba testado de rebote: quando eu testo BuscaInimigo, o construtor e os getters de Inimigo são invocados BuscaInimigo — a classe testada:

```
public class BuscaInimigo {
    InimigoService inimigoService;

    public BuscaInimigo(InimigoService service){ // injeção pelo
construtor
        this.inimigoService = service;
    }

    public Inimigo buscaInimigo(int id){
        String inimigoJson = inimigoService.busca(id); // chama a
dependência
        JsonObject jsonObject =
JsonParser.parseString(inimigoJson).getAsJsonObject();
        return new Inimigo(jsonObject.get("nome").getString(),
                                jsonObject.get("qtdVida").getDouble(),
                                jsonObject.get("arma").getString());
    }
}
```

- O que está sendo testado aqui não é o servidor. É se, dado um JSON, a instância de Inimigo é construída corretamente
- O tipo do campo é a interface InimigoService, não uma classe concreta. É isso que permite injetar o falso InimigoConst — os JSONs hardcoded:

```
public static String SKELETON =
    "{ \"nome\": \"Skeleton\", \"qtdVida\": 200, \"arma\": \"Espada do Skeleton\" }";
```

- Centralizar as constantes em uma classe evita repetir string mágica em cada teste e deixa o dado do teste num lugar só MockInimigoService — o mock manual:

```

public class MockInimigoService implements InimigoService {
    @Override
    public String busca(int id) {
        if (id == 10)      return InimigoConst.SKELETON;
        else if (id == 20) return InimigoConst.DRAGAO;
        else if (id < 0)   return InimigoConst.INEXISTENTE;
        else               return InimigoConst.PADRAO;
    }
    ...
}

```

- Implementa a interface e devolve comportamento trivial, hardcoded
- Fica em src/test porque só faz sentido para o teste. Não é código de produção
- Não tem rede, não tem banco, não tem espera. Roda em microssegundos e sempre devolve a mesma coisa TesteBuscaInimigo (mock manual):

```

@Before
public void setup(){
    service = new MockInimigoService();      // cria o mock
    buscaInimigo = new BuscaInimigo(service); // injeta
}

@Test
public void testeBuscaInimigoSkeleton(){
    Inimigo skeleton = buscaInimigo.buscaInimigo(10);
    assertEquals("Skeleton", skeleton.getNome());
    assertEquals(200.0, skeleton.getQtdVida(), 0.1);
    assertEquals("Espada do Skeleton", skeleton.getArma());
}

```

- Cria o mock, injeta o mock, faz as assertivas. Esse é o roteiro inteiro

Mockito

- Criar mock na mão vira problema quando a classe tem várias dependências, ou quando a interface tem vinte métodos e eu preciso de um só. Eu teria que implementar todos
- Por isso existe framework de mock. O que se espera dele:
 - simular o comportamento da dependência: retornar valor, lançar exceção, modificar parâmetro
 - verificar se as invocações foram corretas: quantas vezes, em que ordem, com quais parâmetros
 - permitir fazer tudo isso dentro do próprio método de teste, sem criar classe nova

- Ganho prático: menos código, teste mais legível, e a configuração do falso fica ao lado da assertiva que depende dela. A mesma classe de teste com Mockito:

```
@RunWith(MockitoJUnitRunner.class)
public class TesteBuscaInimigo {

    @Mock
    private InimigoService service;          // o Mockito instancia isso sozinho
    private BuscaInimigo buscaInimigo;

    @Before
    public void setup(){
        buscaInimigo = new BuscaInimigo(service);
    }

    @Test
    public void testeBuscaInimigoSkeleton(){
        Mockito.when(service.busca(55)).thenReturn(InimigoConst.SKELETON);
        Inimigo skeleton = buscaInimigo.buscaInimigo(55);
        assertEquals("Skeleton", skeleton.getNome());
        ...
    }
}
```

- @Mock: marca o campo. Quem cria a instância falsa é o Mockito
- @RunWith(MockitoJUnitRunner.class): avisa o JUnit que quem comanda a execução é o Mockito, senão os campos @Mock ficam null
- when(chamada).thenReturn(valor): configuro o comportamento. "Quando chamarem busca(55), devolva o JSON do Skeleton"
- Repara que o id virou 55. Com mock manual o id tinha que bater com o if de dentro do MockInimigoService. Com Mockito o id não significa mais nada, é só a chave da configuração. Quem define a resposta é o próprio teste
- Não precisei criar nenhuma classe nova, e não precisei implementar inimigoExistente() para testar busca()
- Por baixo dos panos o Mockito gera uma implementação da interface em tempo de execução e intercepta as chamadas. Por isso ele consegue devolver o que eu mandei e contar quantas vezes o método foi chamado
- Outras coisas do Mockito que aparecem em prova:
 - Mockito.mock(Classe.class): cria mock sem annotation
 - @InjectMocks: injeta os mocks na classe testada automaticamente
 - Mockito.verify(service).busca(10): verifica se o método foi chamado

- Mockito.verify(service, times(2)) / never() / atLeastOnce()
- thenThrow(new RuntimeException()): faz a dependência falhar de propósito, para eu testar o tratamento de erro
- any(), anyInt(), eq(): matchers, quando o valor exato do parâmetro não importa
- A partir do Mockito 2 o MockitoJUnitRunner é estrito: se eu configurar um when() e o teste não usar aquela chamada, ele reclama de UnnecessaryStubbing e falha. É proposital, para não deixar configuração morta no teste

Mock manual x Mockito

	Mock manual	Mockito
Código	uma classe nova por dependência	nenhuma classe nova
Métodos não usados	tenho que implementar todos	ficam com retorno padrão automático
Onde fica a resposta	dentro da classe mock	dentro do próprio teste
Legibilidade	teste limpo, mas a lógica está longe	vejo a configuração e a assertiva juntas
Comportamento complexo	fácil, é Java normal	fica verboso
Verificar interação	tenho que contar na mão	verify() pronto
Curva de aprendizado	zero	precisa aprender a API

- Uso manual quando o falso tem lógica própria de verdade e vai ser reaproveitado por muitos testes (aí ele é mais um fake do que um mock)
- Uso Mockito no resto, que é a maioria dos casos

Teste baseado em estado x baseado em interação

- Baseado em estado: verifica se o código devolveu o resultado esperado, se houve mudança de estado. É o assertEquals do nome do inimigo
- Baseado em interação: verifica se a classe chamou o método da dependência. É o Mockito.verify()
- Existe porque nem toda classe altera estado. Muitas só coordenam, só chamam outras classes. Nessas o único comportamento observável é a chamada
- Na maioria dos casos eu faço teste baseado em estado. Aqui, mesmo tendo interação com InimigoService, o objetivo foi verificar o estado da instância de Inimigo devolvida

Os tipos de dublê de teste

- Dublê (test double) é o termo guarda-chuva para qualquer objeto falso que entra no lugar do real. Os tipos, na classificação do Meszaros/Fowler:
 - dummy: só preenche parâmetro, nunca é usado de verdade
 - stub: devolve resposta pronta e fixa
 - fake: tem implementação de verdade, mas simplificada e imprópria para produção (ex.: banco em memória)
 - spy: é o objeto real, mas registra o que foi chamado
 - mock: configurado com expectativas, e a verificação é sobre as chamadas
- Pegadinha: MockInimigoService se chama Mock mas tecnicamente é um stub/fake, porque ele só devolve valor pronto e ninguém verifica chamada nenhuma
- Na prática o pessoal e os slides chamam tudo de mock. Numa questão discursiva vale citar a distinção

Quando NÃO fazer mock

- Não mockar POJO. Classe que só guarda estado não tem por que ser falsificada, é mais fácil instanciar de verdade
- Não mockar código de terceiro. Eu não tenho controle sobre o comportamento real dele, então meu mock pode estar mentindo. O certo é mockar a minha camada que envolve o terceiro
- Não mockar tudo. Se eu mocko demais, o teste passa a testar os meus mocks e não o sistema. E ele deixa de detectar quando o contrato entre as classes muda
- Por isso teste de mock não substitui teste de integração. Um cobre o buraco do outro

Parte 4 — Rodando de verdade

Rodei as duas classes de teste com os arquivos da pasta, sem alterar nada:

```
[INFO] Running br.inatel.cdg.test.TesteBuscaInimigo
[OK]    testeBuscaInimigoSkeleton
[OK]    testeBuscaInimigoDragao
[OK]    testeBuscaInimigoPadrao
[OK]    testeBuscaInimigoValido
[OK]    testeBuscaInimigoInValido
[INFO] Running br.inatel.cdg.test.mockito.TesteBuscaInimigo
[OK]    testeBuscaInimigoSkeleton
[OK]    testeBuscaInimigoValido
[OK]    testeBuscaInimigoInvalido
```

Tests run: 8, Failures: 0, Errors: 0, Skipped: 0

- Oito testes, todos verdes. A linha "Tests run / Failures / Errors / Skipped" é o formato do relatório do surefire, o que aparece no meio do log do mvn test
- Failure -> a assertiva não bateu. Error -> estourou exceção antes de chegar na assertiva. São coisas diferentes e a prova gosta disso

Um bug que a suíte não pega

- Sondei o mock com vários ids e apareceu isto:

```
id=10    existe=true    busca=Skeleton
id=20    existe=false   busca=Dragao      <- errado
id=9     existe=false   busca=Aranha
id=-8    existe=false   busca=Inexistente
```

- O id 20 devolve o Dragao certinho na busca, mas inimigoExistente(20) devolve false. Olhando o método:

```
for (int i=0; i < list.size(); i++){
    if (list.get(i).equals(id) || list.get(i).equals(id)){
        return true;
    }else{
        return false;    // <- mata o laço na primeira volta
    }
}
```

- O return false dentro do else encerra o laço na iteração 0. Ele nunca chega a comparar com o segundo elemento da lista. Só funciona para o id que estiver na primeira posição
- E a condição está duplicada, o mesmo teste feito duas vezes com `||`
- Correção: tirar o else de dentro do laço e só retornar false depois que o laço terminar. Ou `list.contains(id)`, que resolve em uma linha
- O ponto de engenharia aqui é mais interessante que o bug: a suíte tem 8 testes verdes e mesmo assim o defeito passou. Ninguém testou `verificaArrayListExistente(20)`. Cobertura de linha alta, cobertura de caso baixa
- É o argumento a favor de análise de valor limite: testar 10 (primeiro da lista), 20 (último), 0 (limite entre positivo e negativo), negativo, e um positivo qualquer fora da lista
- Aliás id = 0 devolve Aranha, mas pelo enunciado do exercício zero não é positivo nem negativo. Limite não tratado

Outros cheiros no código

- `verificaArrayListExistente` faz `if (x) return true; else return false;`. É só `return inimigoExistente(id);`
- No teste com Mockito, `testeBuscalnimigoValido` configura o mock e depois chama `service.inimigoExistente(10)` direto, sem passar por `Buscalnimigo`. Ou seja, ele testa o mock, não a classe. Teste que nunca falha por defeito real é teste que não vale nada
- O correto seria chamar `buscalnimigo.verificaArrayListExistente(10)`, que é o método da classe sob teste

Recapitulação

- Maven -> ferramenta de build e gerência de dependência para Java, configurada pelo `pom.xml`, baseada em convenção de diretórios
- GAV -> `groupId`, `artifactId`, `version`. O endereço único de um artefato
- `scope test` -> dependência que só existe para testar e não vai para o jar
- Ciclo default -> `validate`, `compile`, `test`, `package`, `verify`, `install`, `deploy`. Pedir uma fase roda todas as anteriores
- `surefire` -> plugin que roda os testes na fase `test`, e que só enxerga classe com nome `Test/Test`
- Pirâmide de testes -> muita unidade, alguma integração, pouco sistema. Invertida é anti-padrão
- Teste de unidade -> uma classe isolada. Fixture no `@Before`, instância nova por `@Test`
- annotation -> metadado que outro software lê. `@Test` não faz nada sozinha
- Injeção de dependência -> construtor, setter ou parâmetro. Pré-requisito para conseguir mockar
- Mock -> imitação controlada da dependência, para não sair do escopo do teste de unidade
- Mockito -> `when().thenReturn()` para configurar, `verify()` para checar chamada, `@Mock` + `@RunWith` para montar
- Estado x interação -> `assertEquals` no resultado x `verify` na chamada
- Não mockar -> POJO, código de terceiro, e não mockar tudo

Para conferir se entendi

- Por que junit tem `scope test` e gson não?
- O que acontece se eu rodar `mvn package` com um teste falhando, e por que isso é desejável?
- Se `Buscalnimigo` instanciasse `InimigoService` dentro do método, o que exatamente ficaria impossível no teste?
- Por que `InimigoService` precisa ser interface para o esquema funcionar?
- Qual a diferença entre o que `testeBuscalnimigoSkeleton` do pacote `test` e o do pacote `test.mockito` estão fazendo, se os dois passam?

- Escrevendo os testes que faltam para pegar o bug do id 20, quais ids eu escolheria e por quê?
- Em qual situação eu preferiria o mock manual ao Mockito?